

10.1: Parallel and Distributed Systems / DBs



UMD DATA605 - Big Data Systems

10.1: Parallel and Distributed Systems / DBs

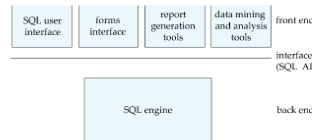
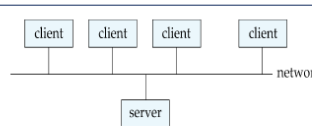
– **Instructor:** Dr. GP Saggese, gsaggese@umd.edu

1 / 16

2 / 16: Client-Server Architecture

Client-Server Architecture

- **Client-server:** Model for distributed applications partitioning tasks between:
 - *Clients:* Request services (e.g., dashboard, GUI, client applications)
 - *Servers:* Provide resources or services (e.g., database)
- **Architecture of a database system:**
 - *Back-end (Server):* Manages access, query evaluation, optimization, concurrency control, and recovery
 - *Front-end (Clients):* Tools like forms, report-writers, GUI
 - Interface between front-end and back-end:
 - SQL
 - Application programming interface (API)



2 / 16

Client-Server Architecture

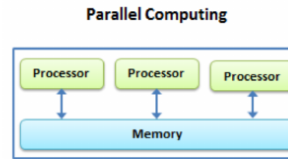
- **Client-server model:** This is a framework for distributed applications where tasks are divided between two main components:
 - **Clients:** These are the entities that request services. Examples include dashboards, graphical user interfaces (GUIs), and client applications. Clients are typically the front-end users interact with.
 - **Servers:** These provide the necessary resources or services, such as databases. Servers handle the back-end processes and respond to client requests.
- **Architecture of a database system:**
 - **Back-end (Server):** This part manages several critical functions:
 - Access management: Ensures only authorized users can access data.
 - Query evaluation and optimization: Processes and improves the efficiency of database queries.
 - Concurrency control: Manages simultaneous operations without conflicts.
 - Recovery: Ensures data integrity and restoration after failures.
 - **Front-end (Clients):** Includes tools that users interact with, such as forms, report-writers, and GUIs.
- **Interface between front-end and back-end:**
 - **SQL:** A standard language for managing and manipulating databases.
 - **Application Programming Interface (API):** Allows different software applications to communicate with each other, facilitating interaction between the client and server.

3 / 16: Parallel vs Distributed Computing

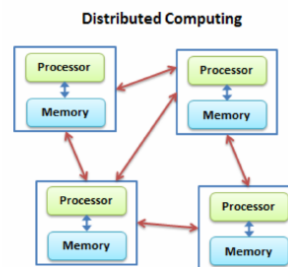
Parallel vs Distributed Computing

– **Parallel computing**

- One computer, multiple CPUs
- Cluster: many computers, each with multiple CPUs
- Homogeneous, geographically close nodes
- Work on one task simultaneously

– **Distributed computing**

- Autonomous, geographically separate systems
- Heterogeneous and distant nodes
- Perform separate tasks or parts of a larger task



3 / 16

– **Parallel Computing**

- *One computer, multiple CPUs:* This setup involves a single machine with several processors working together. Each CPU can handle different parts of a task simultaneously, increasing efficiency.
- *Cluster: many computers, each with multiple CPUs:* A cluster consists of multiple computers, each equipped with several CPUs. These computers are connected and work together as a single system.
- *Homogeneous, geographically close nodes:* The nodes in parallel computing are similar in terms of hardware and are located close to each other, often in the same physical location.
- *Work on one task simultaneously:* All processors focus on a single task, dividing the workload to complete it faster.

– **Distributed Computing**

- *Autonomous, geographically separate systems:* Distributed computing involves independent systems that are spread out over different locations. Each system operates on its own but contributes to a larger task.
- *Heterogeneous and distant nodes:* The nodes can vary in hardware and software configurations and are often located far apart, sometimes across different countries.
- *Perform separate tasks or parts of a larger task:* Each node in a distributed system may handle different tasks or parts of a larger task, working independently but contributing to a common goal.

4 / 16: Parallel Systems

Parallel Systems

- **Parallel systems** consist of:
 - Multiple processors
 - Multiple memories
 - Multiple disks
 - Fast interconnection network
- **Coarse-grain parallel machine**
 - Small number of powerful processors
 - E.g., your laptop with multiple CPUs
- **Fine-grain parallel machine**
 - Aka massively parallel
 - Thousands of smaller processors
 - Larger degree of parallelism
 - With or without shared memory
 - E.g., GPUs, The Connection Machine



The Connection Machine, MIT, 1980s

4 / 16

- **Parallel systems** are computing architectures designed to perform multiple calculations simultaneously. They consist of:
 - **Multiple processors:** These are the brains of the system, allowing for simultaneous data processing.
 - **Multiple memories:** Each processor may have its own memory, or they may share memory, enabling efficient data access.
 - **Multiple disks:** These provide storage for data and programs, allowing for quick access and retrieval.
 - **Fast interconnection network:** This network connects the processors, memories, and disks, ensuring rapid communication and data transfer.
- **Coarse-grain parallel machine:**
 - Involves a **small number of powerful processors**.
 - Example: A typical laptop with multiple CPUs, where each CPU is capable of handling substantial tasks independently.
- **Fine-grain parallel machine:**
 - Also known as **massively parallel** systems.
 - Comprises **thousands of smaller processors**, offering a higher degree of parallelism.
 - Can operate **with or without shared memory**, depending on the architecture.
 - Examples include **GPUs** and historical systems like **The Connection Machine** from the 1980s, which was a pioneering effort in parallel computing.

5 / 16: Parallel Databases: Introduction

Parallel Databases: Introduction

- **Parallel DBs** were the standard approach before MapReduce
- **Parallel machines have become common and affordable**
 - Prices of microprocessors, memory, and disks drop sharply
 - Desktop/laptop computers feature multiple processors
 - Trend will continue
- **DBs are growing increasingly large**
 - Large volumes of transaction data collected and stored for analysis
 - Multimedia objects increasingly stored in databases
- **Large-scale parallel DBs increasingly used for:**
 - Storing large volumes of data
 - Processing time-consuming queries
 - Providing high throughput for transaction processing

5 / 16

- **Parallel Databases: Introduction**
- **Parallel DBs were the standard approach before MapReduce**
 - Before the advent of MapReduce, parallel databases were the go-to solution for handling large-scale data processing tasks. They allowed for the distribution of data and queries across multiple machines, which helped in managing and processing large datasets efficiently.
- **Parallel machines have become common and affordable**
 - The cost of technology components like microprocessors, memory, and disks has decreased significantly over time. This price drop has made it feasible for even personal computers, like desktops and laptops, to have multiple processors. This trend is expected to continue, making parallel computing more accessible to a wider audience.
- **DBs are growing increasingly large**
 - The amount of data being collected and stored is growing rapidly. This includes not only traditional transaction data but also multimedia objects like images and videos. As a result, databases are becoming larger and more complex, necessitating more robust solutions for storage and analysis.
- **Large-scale parallel DBs increasingly used for:**
 - *Storing large volumes of data:* They are essential for managing the vast amounts of data generated daily.

- *Processing time-consuming queries*: Parallel databases can handle complex queries more efficiently by distributing the workload across multiple processors.
- *Providing high throughput for transaction processing*: They ensure that a large number of transactions can be processed quickly and efficiently, which is crucial for businesses that rely on real-time data processing.

6 / 16: Parallel Databases

Parallel Databases

- Internet / Big Data created need for large, fast DBs
 - Store petabytes of data
 - Process thousands of transactions per second (e.g., commerce website)
- **Databases can be parallelized**
 - Set-oriented nature of DB queries suits parallelization
 - Some operations are embarrassingly parallel
 - E.g., join between R and S on $R.b = S.b$ as MapReduce task
- **Parallel DBs**
 - More transactions per second or less time per query
 - Throughput vs response time
 - Speed-up vs scale-up
- **Perfect speedup doesn't happen** due to:
 - Start-up costs
 - Task interference
 - Skew

6 / 16

- **Parallel Databases**
 - The rise of the *Internet* and *Big Data* has led to a demand for databases that are both large and fast. This is because modern applications, like e-commerce websites, need to store vast amounts of data, often measured in petabytes, and handle a high volume of transactions, sometimes thousands per second. This requires databases that can efficiently manage and process such large-scale operations.
- **Databases can be parallelized**
 - The nature of database queries, which are often set-oriented, makes them well-suited for parallel processing. This means that many database operations can be divided into smaller tasks that can be executed simultaneously. Some operations, like certain types of joins, are considered *embarrassingly parallel*, meaning they can be easily split into independent tasks. For example, a join operation between two tables R and S on a common attribute can be efficiently executed using a framework like MapReduce.
- **Parallel DBs**
 - The goal of parallel databases is to either increase the number of transactions processed per second or reduce the time it takes to execute a query. This involves balancing *throughput* (the number of transactions processed in a given time) and *response time* (how quickly a single query is completed). Additionally, there's a focus on achieving

speed-up (how much faster a task can be completed with more resources) and *scale-up* (how well a system can handle a larger workload with more resources).

– **Perfect speedup doesn't happen due to:**

- In practice, achieving perfect speedup is challenging because of several factors. *Start-up costs* refer to the time and resources needed to initiate parallel tasks. *Task interference* occurs when tasks compete for shared resources, leading to delays. *Skew* happens when tasks are unevenly distributed, causing some processors to finish earlier than others, which can lead to inefficiencies. These factors prevent parallel databases from achieving ideal performance improvements.

7 / 16: How to Measure Parallel Performance

How to Measure Parallel Performance

– **Throughput**

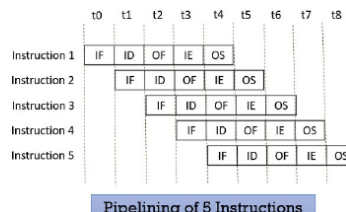
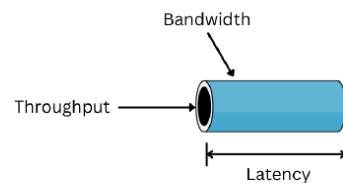
- Number of tasks completed in given time
- Increase by processing tasks in parallel

– **Latency**

- Time to complete single task from submission
- Decrease by performing subtasks in parallel

– **Throughput and latency are related but not the same**

- Increase throughput by reducing latency
- Increase throughput by pipelining (overlapping task execution)
 - E.g., building a car takes weeks, but one car is completed per hour
 - Pipelining of microprocessor



Pipelining of 5 Instructions

7 / 16

– **Throughput**

- *Definition:* Throughput refers to the number of tasks completed in a given time frame. It is a measure of how much work is done.
- *Improvement:* By processing tasks in parallel, throughput can be increased. This means more tasks are completed simultaneously, boosting overall productivity.

– **Latency**

- *Definition:* Latency is the time taken to complete a single task from the moment it is submitted. It measures the delay before the task is finished.
- *Reduction:* Performing subtasks in parallel can decrease latency, allowing tasks to be completed faster.

– **Throughput and Latency Relationship**

- *Connection:* While related, throughput and latency are not the same. Improving one can affect the other.
- *Strategies:*

- Reducing latency can lead to increased throughput.
- Pipelining, or overlapping task execution, is a method to enhance throughput. For example, in car manufacturing, although building a car takes weeks, the assembly line allows one car to be completed every hour. Similarly, pipelining in microprocessors allows multiple instructions to be processed simultaneously, improving efficiency.

8 / 16: Speed-Up and Scale-Up: Intuition

Speed-Up and Scale-Up: Intuition

- **You have a workload to execute**
 - Change workload M
 - Number of DB transactions
 - Amount of DB data to query
- **You need to execute the workload on a machine**
 - Change computing power N
 - Better CPU (scale vertically, scale up)
 - More CPUs (scale horizontally, scale out)
- Two **ways to measure efficiency** when increasing workload and computing power
 - *Speed-up*
 - Keep constant problem size M
 - Increase machine power N
 - *Scale-up*
 - Increase problem size M
 - Increase machine power N

8 / 16

- **Speed-Up and Scale-Up: Intuition**
 - *You have a workload to execute*
 - **Change workload M**
 - This refers to the tasks or operations you need to perform, such as the number of database (DB) transactions or the amount of data you need to query. Essentially, it's about the size or complexity of the job you need to get done.
 - *You need to execute the workload on a machine*
 - **Change computing power N**
 - This involves enhancing the machine's ability to handle tasks. You can do this by upgrading to a better CPU, which is known as scaling vertically or scaling up. Alternatively, you can add more CPUs, which is called scaling horizontally or scaling out. Both methods aim to improve the machine's performance to handle the workload more efficiently.
- **Two ways to measure efficiency** when increasing workload and computing power
 - *Speed-up*
 - This approach focuses on keeping the workload size constant (problem size M) while increasing the machine's computing power (N). The goal is to see how much faster

the workload can be completed with more powerful resources.

– **Scale-up**

- Here, both the workload size (M) and the machine's computing power (N) are increased. The aim is to understand how well the system can handle larger tasks as both the problem and the resources grow. This is crucial for systems that need to manage increasing amounts of data or transactions over time.

9 / 16: Speed-Up vs Scale-Up

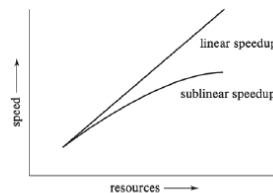
Speed-Up vs Scale-Up

- The amount of computing power N can be changed
- The amount of work M can be changed

- **Speed-up**: fixed-sized problem on a small system given to a system N -times larger

$$\text{speed-up} = \frac{\text{small system elapsed time}}{\text{large system elapsed time}}$$

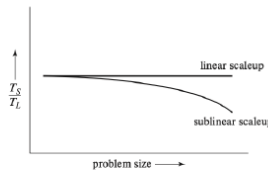
- Speed-up is linear if equation equals N



- **Scale-up**: increase size of both problem M and system N
 - N -times larger system to perform M -times larger job

$$\text{scale-up} = \frac{\text{small system-problem time}}{\text{big system-problem time}}$$

- Scale-up is linear if equation equals 1



9 / 16

– **Speed-Up vs Scale-Up**

- These concepts relate to how we can improve computing performance by adjusting resources or workload.

– **Speed-Up**

- *Definition*: Speed-up involves solving a fixed-sized problem on a larger system.
- *Formula*: $\text{speed-up} = \frac{\text{small system elapsed time}}{\text{large system elapsed time}}$
- *Linear Speed-Up*: Achieved when the speed-up equals the increase in resources (N). This means doubling resources halves the time.
- *Graph Explanation*: The graph shows linear speed-up as a straight line, indicating proportional improvement with added resources. Sublinear speed-up curves below, showing diminishing returns.

– **Scale-Up**

- *Definition*: Scale-up involves increasing both the problem size and the system size.
- *Formula*: $\text{scale-up} = \frac{\text{small system-problem time}}{\text{big system-problem time}}$
- *Linear Scale-Up*: Achieved when the scale-up ratio equals 1, meaning the system handles larger problems efficiently.
- *Graph Explanation*: The graph shows linear scale-up as a flat line, indicating consistent

performance with increased problem size. Sublinear scale-up curves downward, indicating less efficiency as problems grow.

These concepts are crucial in understanding how to optimize computing systems for better performance and efficiency.

10 / 16: Factors Limiting Speed-up and Scale-up

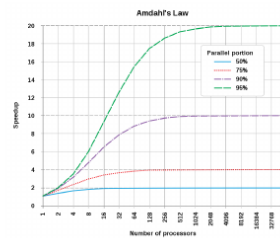
Factors Limiting Speed-up and Scale-up

- **Speed-up and scale-up are typically sub-linear** due to several issues
 - E.g., not all computation is parallelizable

- **Amdahl's Law**

- p = fraction parallelizable
- s = number of nodes
- T = execution time serially
- $T(p)$ = execution time on s nodes = $(1 - p)T + (p/s)T$

$$Speedup(s) = \frac{T}{T(s)} = \frac{1}{(1 - p) + \frac{p}{s}}$$



- E.g.,
 - 90% parallelizable → max speed-up 10x
 - 50% parallelizable → max speed-up 2x (even with infinite nodes)

10 / 16

- **Factors Limiting Speed-up and Scale-up**

- *Speed-up* and *scale-up* are often less than expected because not all tasks can be done in parallel. This means that even if you add more resources, the improvement in performance might not be as large as anticipated.

- **Amdahl's Law**

- This law helps us understand the limits of parallel computing. It shows how the fraction of a task that can be parallelized affects overall speed-up.

- **Variables:**

- p : The portion of the task that can be parallelized.
- s : The number of nodes or processors used.
- T : The time it takes to complete the task serially (without parallelization).
- $T(p)$: The time it takes to complete the task using s nodes.

- **Formula:**

- The speed-up is calculated as:

$$Speedup(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

- **Examples:**

- If 90% of a task is parallelizable, the maximum speed-up is 10 times, regardless of how many nodes are used.
- If only 50% is parallelizable, the maximum speed-up is just 2 times, even with infinite nodes.
- **Graph Explanation**
 - The graph illustrates how different levels of parallelization affect speed-up as the number of processors increases.
 - As the parallel portion increases, the potential speed-up also increases, but it eventually levels off, showing diminishing returns.

11 / 16: Factors Limiting Speed-up and Scale-up

Factors Limiting Speed-up and Scale-up

- **Startup costs**
 - Starting processes may dominate computation time
 - E.g., dBs create thread pool at startup
- **Interference**
 - Processes compete for shared resources (e.g., system bus, disks, locks)
 - Time spent waiting on other processes
 - E.g., devs touching same code create merge conflicts
- **Cost of synchronization**
 - Smaller work pieces increase synchronization complexity
 - E.g., hiring many developers in a company
- **Skew**
 - Splitting work increases variance in task response time
 - Difficult to split tasks equally
 - Execution time determined by slowest task

11 / 16

- **Startup costs**
 - When we talk about *startup costs*, we're referring to the initial time and resources needed to get a process up and running. This can be a significant factor because sometimes the time it takes to start a process can be longer than the time it takes to actually perform the task. For example, databases often create a thread pool when they start up, which can take a while and delay the actual processing of data.
- **Interference**
 - *Interference* occurs when multiple processes are trying to use the same resources at the same time. This can lead to delays because processes have to wait for others to finish using shared resources like the system bus, disks, or locks. A real-world analogy is when developers work on the same piece of code, leading to merge conflicts that need to be resolved before progress can continue.
- **Cost of synchronization**
 - When tasks are broken down into smaller pieces, the need for *synchronization* increases.

This means more time and effort are required to coordinate these tasks. For instance, if a company hires many developers, it becomes more complex to manage and synchronize their work, which can slow down overall progress.

– **Skew**

- *Skew* refers to the uneven distribution of work among tasks, which can lead to some tasks taking much longer than others. This is problematic because the overall execution time is often determined by the slowest task. It's challenging to divide tasks perfectly evenly, and this variance can significantly impact the efficiency of a process.

12 / 16: Topology of Parallel Systems

Topology of Parallel Systems

- Many ways to organize computation and storage
 - M = memory
 - P = processors
 - D = disks
- **Topology**
 - Shared memory
 - Shared disk
 - Shared nothing
 - Hierarchical
- **Problems**
 - Cache coherency
 - Data communication
 - Fault tolerance
 - Resource congestion

(a) shared memory

(b) shared disk

(c) shared nothing

(d) hierarchical

12 / 16

– **Many ways to organize computation and storage**

- In parallel systems, computation and storage can be organized in various ways using components like memory (M), processors (P), and disks (D).

– **Topology**

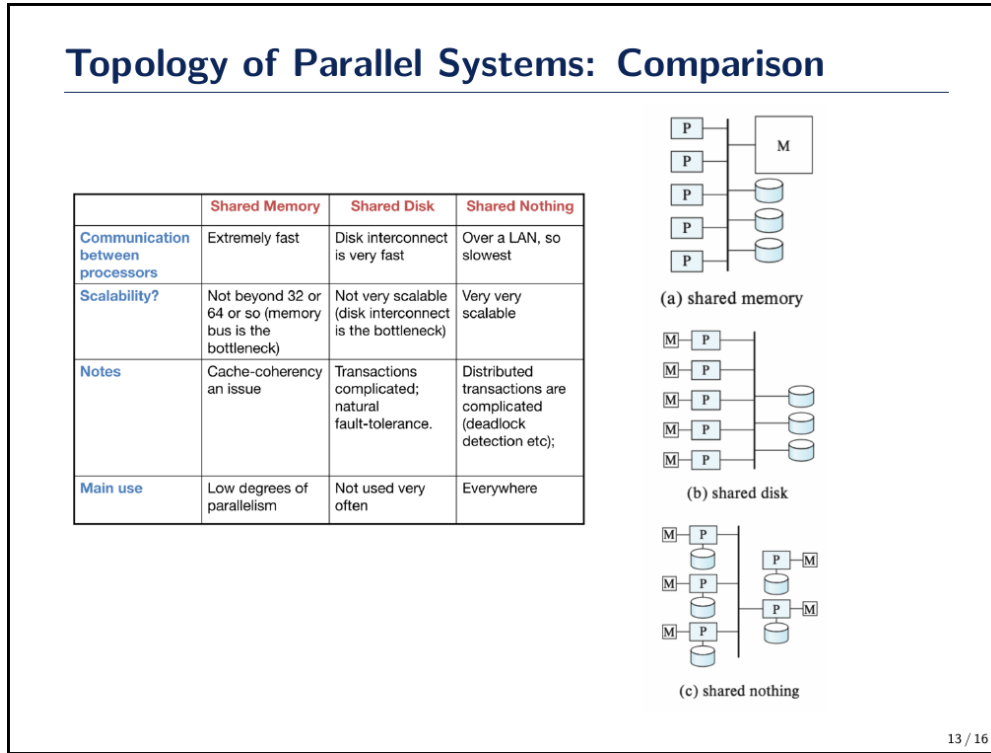
- *Shared memory*: All processors access a common memory space. This setup can simplify programming but may lead to bottlenecks.
- *Shared disk*: Each processor has its own memory, but disks are shared. This allows for data sharing while maintaining some independence.
- *Shared nothing*: Each processor has its own memory and disk. This setup maximizes independence and scalability but requires efficient data distribution.
- *Hierarchical*: Combines elements of the above topologies, often used to balance performance and resource sharing.

– **Problems**

- *Cache coherency*: Ensuring that all processors have the most recent data in their caches.
- *Data communication*: Efficiently transferring data between processors.

- *Fault tolerance*: Maintaining system functionality despite failures.
- *Resource congestion*: Avoiding bottlenecks in resource access, particularly in shared systems.

13 / 16: Topology of Parallel Systems: Comparison



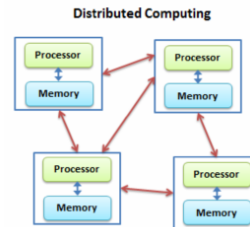
- **Shared Memory**
 - *Communication between processors*: Extremely fast due to direct access to a common memory space.
 - *Scalability*: Limited to around 32 or 64 processors because the memory bus becomes a bottleneck.
 - *Notes*: Cache-coherency is a significant issue, as multiple processors need to maintain a consistent view of memory.
 - *Main use*: Suitable for low degrees of parallelism where fast communication is crucial.
- **Shared Disk**
 - *Communication between processors*: Fast, but relies on a disk interconnect.
 - *Scalability*: Not very scalable; the disk interconnect limits performance as more processors are added.
 - *Notes*: Transactions are complicated, but there is natural fault-tolerance due to shared disk access.
 - *Main use*: Not commonly used due to complexity and scalability issues.
- **Shared Nothing**
 - *Communication between processors*: Slowest, as it occurs over a LAN.
 - *Scalability*: Highly scalable, as each processor has its own memory and disk.
 - *Notes*: Distributed transactions are complex, involving challenges like deadlock detection.

- *Main use:* Widely used in systems requiring high scalability and independence between nodes.

14 / 16: Distributed Databases

Distributed Databases

- **Distributed DBs**
 - DB stored on nodes at geographically separated sites
 - Communicate through high-speed private networks or Internet
- **Why needed?**
 - Large corporation with global offices
 - Redundancy and disaster recovery
 - Natural disasters, power outage, hacker attacks
 - Achieve high-availability despite failures
 - Typically not for performance
 - Use parallel DB for high performance
- **Wide-area networks (WAN) vs Local-area networks (LAN)**
 - Lower bandwidth
 - Higher latency
 - Greater probability of failures and network partition
 - No sharing of memory or disks
 - Communication delay is dominant



14 / 16

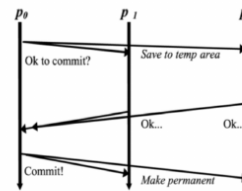
- **Distributed DBs**
 - *Definition:* Distributed databases are systems where the database is stored across multiple nodes located in different geographical locations. These nodes communicate via high-speed private networks or the Internet.
 - *Purpose:* They are essential for large corporations with global offices to ensure data redundancy and disaster recovery. This setup helps maintain high availability even during failures like natural disasters, power outages, or hacker attacks. However, distributed databases are not primarily designed for performance enhancement; parallel databases are more suited for that purpose.
- **Why needed?**
 - *Global Reach:* Useful for corporations with offices worldwide.
 - *Redundancy:* Provides backup and recovery options.
 - *High Availability:* Ensures continuous operation despite failures.
 - *Performance Note:* Not typically used for performance improvements; parallel databases are better for this.
- **Wide-area networks (WAN) vs Local-area networks (LAN)**
 - *Bandwidth and Latency:* WANs have lower bandwidth and higher latency compared to LANs, leading to a higher chance of failures and network partitioning.
 - *Memory and Disk Sharing:* There is no sharing of memory or disks in WANs, making communication delays more significant.
 - *Node Differences:* Nodes in distributed databases can vary in size and function, unlike

parallel databases where nodes are usually similar.

15 / 16: Consistency Issues in Distributed DB Systems

Consistency Issues in Distributed DB Systems

- **Parallel and distributed DBs**
 - Efficient for query processing (read-only)
 - Requires consistency enforcement
- **Atomicity issues**
 - *Problem:* Transaction is all-or-nothing across nodes
 - Two-phase commit (2PC) is centralized
 - Commit decision by a single coordinator node
 - Each node executes transaction, reaching “ready state”
 - If all nodes reach ready state, coordinator commits
 - If a node fails in ready state, it can recover (e.g., write-ahead logs)
 - If a node aborts, coordinator aborts transaction
 - Distributed consensus, e.g.,
 - Paxos
 - Blockchain



Coordinator :

- multicast: *ok to commit?*
- collect replies
- all *ok* => *send commit*
- else => *send abort*

Participant:

- ok to commit* =>
- save to temp area, reply *ok*
- commit* =>
- make change permanent
- abort* =>
- delete temp area

15 / 16

- **Parallel and distributed DBs**
 - These databases are designed to handle large-scale data processing efficiently, especially for read-only queries. However, maintaining consistency across distributed nodes is crucial to ensure data integrity and reliability.
- **Atomicity issues**
 - The main challenge is ensuring that a transaction is completed entirely or not at all across different nodes. This is known as the all-or-nothing principle.
- **Two-phase commit (2PC)**
 - This is a protocol used to ensure atomicity in distributed systems. It involves a centralized approach where a single coordinator node makes the commit decision.
 - Each participating node executes the transaction and reaches a “ready state.” If all nodes are ready, the coordinator commits the transaction.
 - If a node fails while in the ready state, it can recover using mechanisms like write-ahead logs. If any node aborts, the coordinator will abort the entire transaction.
- **Distributed consensus**
 - Protocols like Paxos and blockchain are used to achieve consensus across distributed systems, ensuring that all nodes agree on the transaction outcome. These methods help in maintaining consistency and reliability in distributed databases.

16 / 16: Consistency Issues in Distributed DB Systems

Consistency Issues in Distributed DB Systems

- **Concurrency issues**
 - *Problem:* Multiple processes writing and reading simultaneously
 - Locks / deadlock management
- **Autonomy issues**
 - *Problem:* Units/departments protective of their systems
 - E.g., administering systems, patching, updating

16 / 16

- **Consistency Issues in Distributed DB Systems**
- **Concurrency issues**
 - *Problem:* When multiple processes are writing and reading data at the same time, it can lead to inconsistencies. This is because one process might read data that is being modified by another process, leading to incorrect or unexpected results.
 - To manage this, systems use locks to control access to data. However, this can lead to deadlocks, where two or more processes are waiting for each other to release locks, causing the system to halt. Managing these locks and preventing deadlocks is crucial for maintaining consistency.
- **Autonomy issues**
 - *Problem:* Different units or departments within an organization often want to maintain control over their own systems. This can create challenges when trying to ensure consistency across a distributed database system.
 - For example, each unit might have its own schedule for administering systems, applying patches, or updating software. This lack of coordination can lead to inconsistencies and difficulties in maintaining a unified system. It's important to balance the need for autonomy with the need for consistency in a distributed environment.